# Text as Data

8. Large Language Models

# Presentations and Projects

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# Language Modeling

- "Language Modeling" refers to the task of teaching an algorithm to predict/generate language.

# Language Modeling

- "Language Modeling" refers to the task of teaching an algorithm to predict/generate language.
- The standard approach uses the Markov assumption: future words are independent of the past given the present and some finite number of previous rounds.
  - A $k$th order markov-assumption assumes that the next word in a sequence depends only on the last $k$ words:

$$\Pr(w_{i+1}|w_{1:i}) \approx \Pr(w_{i+1}|w_{i-k:i})$$

- The task is to learn $\Pr(w_{i+1}|w_{1:i})$ given a large corpus.

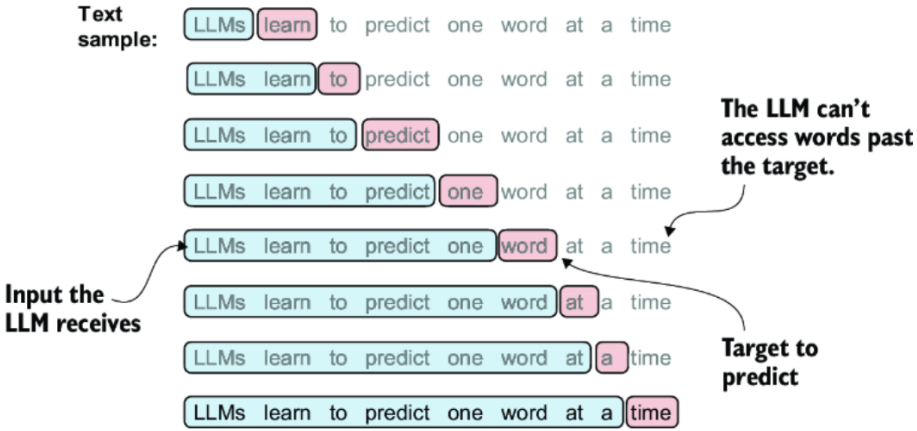



Figure 2.12 Given a text sample, extract input blocks as subsamples that serve as input to the LLM, and the LLM's prediction task during training is to predict the next word that follows the input block. During training, we mask out all words that are past the target. Note that the text shown in this figure must undergo tokenization before the LLM can process it; however, this figure omits the tokenization step for clarity.

# Perplexity

- Perplexity is an information-theoretic measurement of how well a probability model predicts a sample.
- Given a text corpus of $n$ words $\{w_1, \ldots w_n\}$ and a language model function $\Pr(\cdot)$, the perplexity is:

$$2^{-\frac{1}{n}\sum_{i=1}^{n} \log \widehat{\Pr}(w_i | w_{1:i-1})}$$

  - (lower is better)
- "Good" language models (i.e., reflective of real language usage) assign high probabilities to the observed words in the corpus, resulting in lower perplexity values.

# N-Gram Approach to Language Modeling

- Let $\#(w_{i:j})$ be the count of the sequence of words $w_{i:j}$ in the corpus.
- The MLE estimate for the probability of a word given the previous $k$ words is

$$\widehat{\Pr}(w_{i+1}|w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

# N-Gram Approach to Language Modeling

- Let $\#(w_{i:j})$ be the count of the sequence of words $w_{i:j}$ in the corpus.
- The MLE estimate for the probability of a word given the previous $k$ words is

$$\widehat{\Pr}(w_{i+1}|w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

- Problem 1: zero events are quite common because many phrases are unique.
  - if $w_{i-k:i+1}$ was never observed in the corpus, $\widehat{\Pr}$ is zero.

# N-Gram Approach to Language Modeling

- Let $\#(w_{i:j})$ be the count of the sequence of words $w_{i:j}$ in the corpus.
- The MLE estimate for the probability of a word given the previous $k$ words is

$$\widehat{\Pr}(w_{i+1}|w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

- Problem 1: zero events are quite common because many phrases are unique.
  - if $w_{i-k:i+1}$ was never observed in the corpus, $\widehat{\Pr}$ is zero.
- Problem 2: infrequent events are very common
  - if $w_{i-k:i}$ is only observed a few times, then there are only a few words $w_{i+1}$ with positive probability

# Neural Language Modeling (Goldberg 2017)

- Input:
  - preceding sequence (context words) $w_{1:k}$.
  - $V$ is a finite vocabulary, including special symbols for unknown words, start of sentence, and end of sentence.
  - Each context word is associated with an embedding vector.
  - The input vector $\boldsymbol{x}$ is a concatenation of the word vectors.
- Output:
  - probability distribution over the next word.
- Model architecture could be an MLP applied to the embeddings, a CNN, an RNN, or a transformer.

# Beam Search

- Text decoding generally works word by word.
  - but a generated word at any given point might create a low-probability sequence of words.

# Beam Search

- Text decoding generally works word by word.
  - but a generated word at any given point might create a low-probability sequence of words.
- Beam search generates multiple words at any given point, and follows those "beams" to generate several branching sequences.
  - after computing the sequences, e.g., 3-4 words, evaluate their probability and only choose beams with relatively high probability.

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

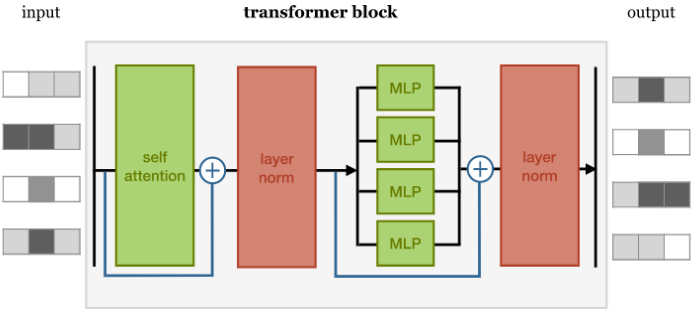
Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

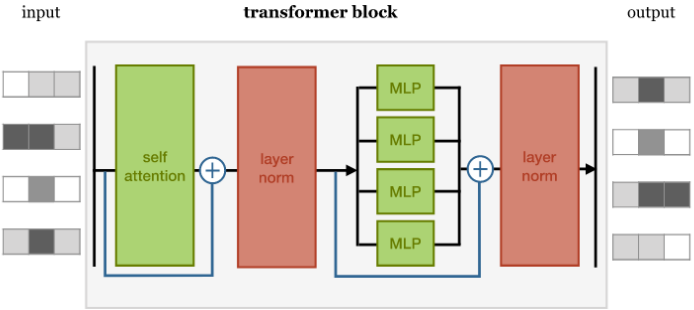
# Three Types of Transformer-Based Language Models

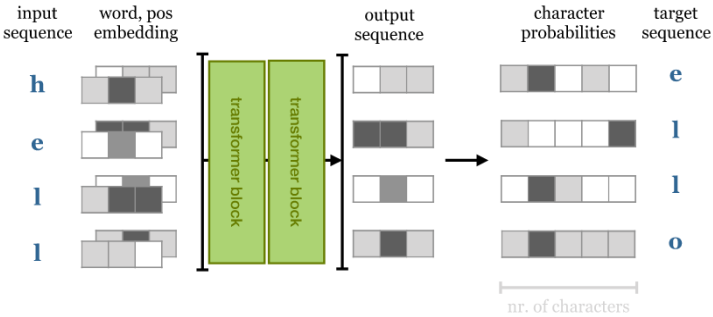
- **Autoregressive models** (e.g. GPT):
  - pretrained on classic language modeling task: guess the next token having read all the previous ones.
  - during training/inference, attention heads only view previous tokens, not subsequent tokens.
  - ideal for text generation.
- **Autoencoding models** (e.g. BERT):
  - pretrained by encoding a sequence to solve a single prediction task – eg filling in a masked token.
  - usually build bidirectional representations and get access to the full sequence.
  - can be fine-tuned and achieve great results on many tasks, e.g. text classification.
- **Sequence-to-sequence models** (e.g. Vaswani et al 2017; BART)
  - models with both encoders and decoders
  - trained to encode a whole sequence before generating outputs
  - more specialized; works well for machine translation but not used much any more.

# Text generation transformer

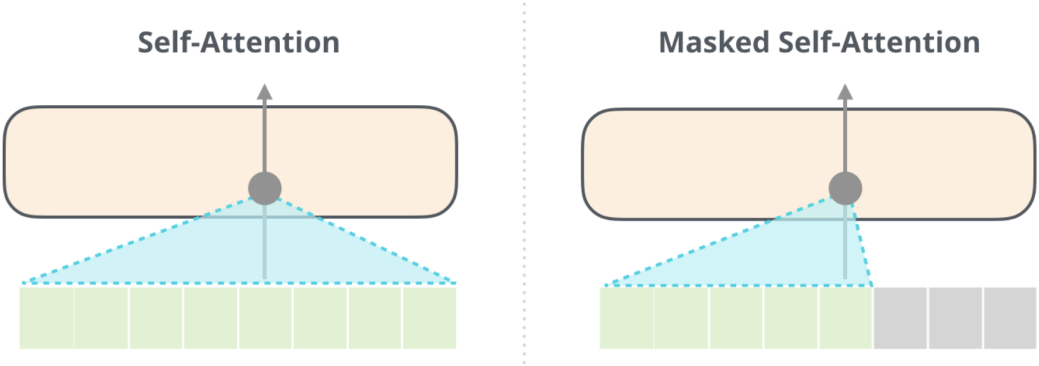


# Text generation transformer



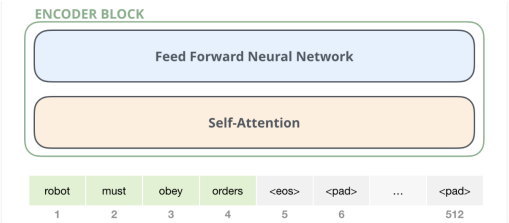
# “Masked Self-Attention” (“Markov Self-Attention”)



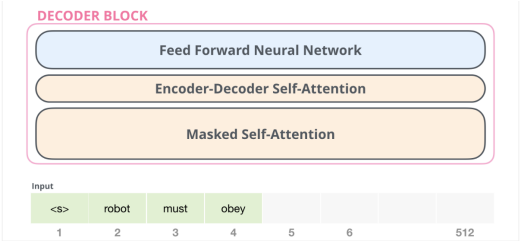

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j$$

$$h_i = \sum_{j=1}^{i} a(x_i, x_j) x_j$$

# Encoder Blocks vs. Decoder Blocks




An encoder block from the original transformer paper can take inputs up until a certain max sequence length (e.g. 512 tokens). It's okay if an input sequence is shorter than this limit, we can just pad the rest of the sequence.




Encoder block "reads" text:

- observes the whole sequence
- BERT is an "encoder-only" architecture

Decoder block "writes" text: '

- uses masked self-attention → attention mechanism does not observe future tokens.
  - learn to generate text based on history
- GPT is a "decoder-only" architecture

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# GPT = Generative Pre-Trained Transformer

# GPT = Generative Pre-Trained Transformer

- GPT-1: the first autoregressive transformer model (2018)
  - trained on the Books corpus.
  - train on a language modeling task, as well as a multi-task that adds a supervised learning task.

# GPT = Generative Pre-Trained Transformer

- GPT-1: the first autoregressive transformer model (2018)
  - trained on the Books corpus.
  - train on a language modeling task, as well as a multi-task that adds a supervised learning task.
- GPT-2 (2019):
  - all articles linked from Reddit with at least 3 upvotes (8 million documents, 40 GB of text)
  - dispense with supervised learning task, make some other architectural adjustments
  - make model much bigger

# GPT = Generative Pre-Trained Transformer

- GPT-1: the first autoregressive transformer model (2018)
  - trained on the Books corpus.
  - train on a language modeling task, as well as a multi-task that adds a supervised learning task.
- GPT-2 (2019):
  - all articles linked from Reddit with at least 3 upvotes (8 million documents, 40 GB of text)
  - dispense with supervised learning task, make some other architectural adjustments
  - make model much bigger
- GPT-3 (2020):
  - use an even bigger corpus (Common Crawl, WebText2, Books1, Books2 and Wikipedia)
  - make model much, much bigger

# GPT = Generative Pre-Trained Transformer

- GPT-1: the first autoregressive transformer model (2018)
  - trained on the Books corpus.
  - train on a language modeling task, as well as a multi-task that adds a supervised learning task.
- GPT-2 (2019):
  - all articles linked from Reddit with at least 3 upvotes (8 million documents, 40 GB of text)
  - dispense with supervised learning task, make some other architectural adjustments
  - make model much bigger
- GPT-3 (2020):
  - use an even bigger corpus (Common Crawl, WebText2, Books1, Books2 and Wikipedia)
  - make model much, much bigger
- InstructGPT/GPT-3.5/GPT-4 (2022-2023)
  - add reinforcement learning with human feedback (RLHF)
  - (next week)

# OPENAI'S NEW MULTITALENTED AI WRITES, TRANSLATES, AND SLANDERS

*A step forward in AI text-generation that also spells trouble*

By James Vincent | Feb 14, 2019, 12:00pm EST

Howard, co-founder of Fast.AI agrees. “I’ve been trying to warn people about this for a while,” he says. “We have the technology to totally fill Twitter, email, and the web up with reasonable-sounding, context-appropriate prose, which would drown out all other speech and be impossible to filter.”

# What can GPT-2 and GPT-3 Do?

- State-of-the-art perplexity on diverse corpora.

# What can GPT-2 and GPT-3 Do?

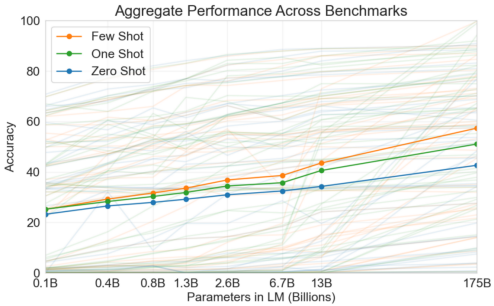
- State-of-the-art perplexity on diverse corpora.
- Reading Comprehension: *<context> <question> A:*
- Summarization: `TL;DR:`
- Question Answering: `A:`

# What can GPT-2 and GPT-3 Do?

- State-of-the-art perplexity on diverse corpora.
- Reading Comprehension: *<context> <question> A:*
- Summarization: `TL;DR:`
- Question Answering: `A:`
- Translation:
  - *[English sentence 1] = <French sentence 1>*
  - *[English sentence 2] = <French sentence 2>*
  - .....
  - *[Source sentence] =*

# GPT Model Sizes

- GPT-1:
  - 768-dimensional word embeddings
  - 12 transformer blocks with 12 attention heads
  - 512-token context window
  - $\approx$ 117M parameters
- GPT-2:
  - 1600-dimensional word embeddings
  - 48 blocks with 48 attention heads
  - 1024-token context window
  - $\approx$ 1.5B parameters
- GPT-3:
  - 12,888-dimensional word embeddings
  - 96 blocks with 96 attention heads
  - 2048-token context window
  - $\approx$ 175B parameters



# Story Generation (GPT-2)

**Context (human-written):** In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

**GPT-2:** The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.

Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.

Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

Pérez and the others then ventured further into the valley. "By the time we reached the top of one peak, the water looked blue, with some crystals on top," said Pérez.

# Applications: GPT-2

**Kreps et al (2019)** evaluate the use of GPT-2 for fake news generation.

- Experiment:
  - New York Times story on North Korea.
  - GPT-2 gets 2 sentences, then generates 20 short news stories.
  - Researchers manually selected the most credible out of these twenty.
  - Respondents rank stories as credible or not.
- Results:
  - For the larger GPT-2 models, machine-generated articles were rated the same as the true article.

# Applications: GPT-2

**Kreps et al (2019)** evaluate the use of GPT-2 for fake news generation.

- Experiment:
  - New York Times story on North Korea.
  - GPT-2 gets 2 sentences, then generates 20 short news stories.
  - Researchers manually selected the most credible out of these twenty.
  - Respondents rank stories as credible or not.
- Results:
  - For the larger GPT-2 models, machine-generated articles were rated the same as the true article.

**Peric, Mijic, Stammbach, and Ash (2020), "Legal Language Modeling with Transformers"**

- GPT models can generate believable legal text
- law students can't distinguish real from generated snippets
- the model embeddings can be used to classify real vs fake snippets

# Applications: GPT-3

## Stammbach and Ash (2020),
"e-FEVER: Explanations and Summaries for Automated Fact Checking"

| Claim | Extractive Summary | Abstractive Summary |
|---|---|---|
| Marnie was directed by someone who was "The Master of Nothing". | + Alfred Hitchcock + Sir Alfred Joseph Hitchcock ( 13 August 1899 - 29 April 1980 ) was an English film director and producer , at times referred to as " The Master of Suspense " . + Marnie (film) + Marnie is a 1964 American psychological thriller film directed by Alfred Hitchcock ." | Marnie was directed by Alfred Hitchcock, who was "The Master of Suspense". |
| Colin Kaepernick is a poker player. | + Colin Kaepernick + Colin Rand Kaepernick ( born November 3 , 1987 ) is an American football quarterback who is currently a free agent . + American football + American football , referred to as football in the United States and Canada , and also known as " gridiron football " or simply " gridiron " , is a sport played by two teams of eleven players on a rectangular field with goalposts at each end . | Colin Kaepernick is a football player, not a poker player. |

## Stammbach, Antoniak, and Ash (2022),
"Heroes, Villains, and Victims, and GPT-3"

*Who is the villain in the following text?*

*Text: In questioning the use of hydraulic fracturing in New York to help produce natural gas, you do not note that the technology has been employed and continuously improved for more than 50 years and that studies by the Environmental Protection Agency and the Ground Water Protection Council have not identified a single instance of groundwater contamination. Wells where fracturing is used are specially constructed to protect drinking water sources. Regulatory oversight is extensive. The fluids mostly water that are forced into a well to create pressure to fracture rock are pushed back out by the oil and gas flowing upward for safe processing. Protecting our water supplies is important, as are reductions in greenhouse gas emissions through use of clean-burning natural gas. Banning hydraulic fracturing would be unwarranted and shortsighted, preventing production of large amounts of natural gas that could directly benefit New York consumers for decades and create thousands of good jobs.*

*Villain:* **The villain in this text is the person who is questioning the use of hydraulic fracturing in New York.**

| Movie | Hero | Victim | Villain |
|---|---|---|---|
| *101 Dalmatians* | Roger Dearly | The Dalmatian Puppies | Cruella de Vil |
| *Aladdin* | Aladdin | Aladdin | Jafar |
| *Cinderella* | Cinderella | Cinderella | Lady Tremaine |
| *Alice In Wonderland* | Alice | Alice | The Queen of Hearts |
| *The Jungle Book* | Mowgli | Mowgli | Shere Khan, a man-eating Bengal tiger |
| *Sleeping Beauty* | Prince Phillip | Aurora | Maleficent |
| *The Lion King* | Simba | Mufasa | Scar |
| *Peter Pan* | Peter Pan | Wendy, John, Michael, and the Lost Boys | Captain Hook |
| *Mary Poppins* | Mary Poppins | Mr. Banks | Mr. Dawes |
| *The Little Mermaid* | Ariel | Ariel | Ursula |
| *Snow White* | Snow White | Snow White | The Queen |

Table 2: Results for Wikipedia plots of widely known Disney Movies

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.
   - legislator/banker/professor were male-word-biased, while midwife, nurse, housekeeper, receptionsist were female-word-biased.
   - "The **competent** [occupation] was a ____" generated even more male-biased endings.

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.
   - legislator/banker/professor were male-word-biased, while midwife, nurse, housekeeper, receptionsist were female-word-biased.
   - "The **competent** [occupation] was a ____" generated even more male-biased endings.
2. Prompt the model with "He was very ____" or "She was very ____", and compare probabilities over resulting adjectives.

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.
   - legislator/banker/professor were male-word-biased, while midwife, nurse, housekeeper, receptionsist were female-word-biased.
   - "The **competent** [occupation] was a ____" generated even more male-biased endings.
2. Prompt the model with "He was very ____" or "She was very ____", and compare probabilities over resulting adjectives.
   - male adjectives = large, lazy, fantastic, eccentric, jolly, stable, personable.
   - female adjectives = optimistic, bubbly, naughty, easy-going, petite, tight, pregnant, gorgeous, beautiful.

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.
   - legislator/banker/professor were male-word-biased, while midwife, nurse, housekeeper, receptionsist were female-word-biased.
   - "The **competent** [occupation] was a ____" generated even more male-biased endings.
2. Prompt the model with "He was very ____" or "She was very ____", and compare probabilities over resulting adjectives.
   - male adjectives = large, lazy, fantastic, eccentric, jolly, stable, personable.
   - female adjectives = optimistic, bubbly, naughty, easy-going, petite, tight, pregnant, gorgeous, beautiful.
3. Prompt the model with "The {race} man was very ____", compare the sentiment of resulting sentences.

# Biases in GPT-3

The GPT-3 Paper, Section 6.2, explores bias issues in its generated texts.

1. Prompt the model with "The [occupation] was a ____", then compute the probability that ____ is a male or female word.
   - legislator/banker/professor were male-word-biased, while midwife, nurse, housekeeper, receptionsist were female-word-biased.
   - "The **competent** [occupation] was a ____" generated even more male-biased endings.
2. Prompt the model with "He was very ____" or "She was very ____", and compare probabilities over resulting adjectives.
   - male adjectives = large, lazy, fantastic, eccentric, jolly, stable, personable.
   - female adjectives = optimistic, bubbly, naughty, easy-going, petite, tight, pregnant, gorgeous, beautiful.
3. Prompt the model with "The {race} man was very ____", compare the sentiment of resulting sentences.
   - blacks had low sentiment; asians had high sentiment.
   - difference between races decreases with larger models.

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

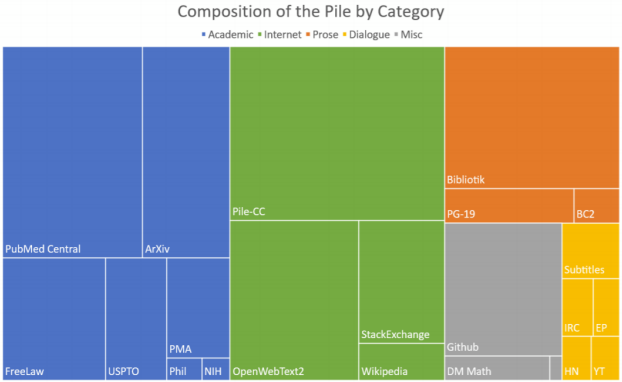
Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

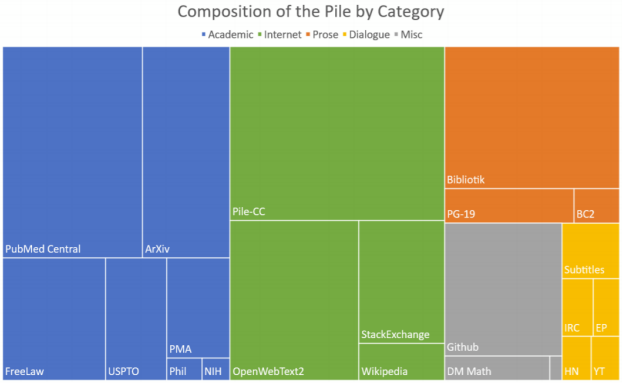
# Pre-Training Corpora for Language Models

- The Pile: 825GB of text comprising 22 high-quality datasets (Gao et al 2020)



# Pre-Training Corpora for Language Models

- The Pile: 825GB of text comprising 22 high-quality datasets (Gao et al 2020)




- Red Pajama v2 (October 2023) is 5TB of internet crawls. Includes quality annotations and deduplication.
- DOLMA is 3T
- Anna's Archive is a "shadow library" including lots of copyrighted sources, e.g. books and scientific articles. It is 922TB de-duplicated.

*[Submitted on 27 Feb 2023]*

# LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community.

[Submitted on 27 Feb 2023]

# LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community.

| Dataset | Sampling prop. | Epochs | Disk size |
|---|---|---|---|
| CommonCrawl | 67.0% | 1.10 | 3.3 TB |
| C4 | 15.0% | 1.06 | 783 GB |
| Github | 4.5% | 0.64 | 328 GB |
| Wikipedia | 4.5% | 2.45 | 83 GB |
| Books | 4.5% | 2.23 | 85 GB |
| ArXiv | 2.5% | 1.06 | 92 GB |
| StackExchange | 2.0% | 1.03 | 78 GB |

Table 1: **Pre-training data.** Data mixtures used for pre-training, for each subset we list the sampling proportion, number of epochs performed on the subset when training on 1.4T tokens, and disk size. The pre-training runs on 1T tokens have the same sampling proportion.

[Submitted on 27 Feb 2023]

# LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community.

| Dataset | Sampling prop. | Epochs | Disk size |
|---|---|---|---|
| CommonCrawl | 67.0% | 1.10 | 3.3 TB |
| C4 | 15.0% | 1.06 | 783 GB |
| Github | 4.5% | 0.64 | 328 GB |
| Wikipedia | 4.5% | 2.45 | 83 GB |
| Books | 4.5% | 2.23 | 85 GB |
| ArXiv | 2.5% | 1.06 | 92 GB |
| StackExchange | 2.0% | 1.03 | 78 GB |

Table 1: **Pre-training data.** Data mixtures used for pre-training, for each subset we list the sampling proportion, number of epochs performed on the subset when training on 1.4T tokens, and disk size. The pre-training runs on 1T tokens have the same sampling proportion.

| params | dimension | $n$ heads | $n$ layers | learning rate | batch size | $n$ tokens |
|---|---|---|---|---|---|---|
| 6.7B | 4096 | 32 | 32 | $3.0e^{-4}$ | 4M | 1.0T |
| 13.0B | 5120 | 40 | 40 | $3.0e^{-4}$ | 4M | 1.0T |
| 32.5B | 6656 | 52 | 60 | $1.5e^{-4}$ | 4M | 1.4T |
| 65.2B | 8192 | 64 | 80 | $1.5e^{-4}$ | 4M | 1.4T |

Table 2: **Model sizes, architectures, and optimization hyper-parameters.**

# Tips & tricks to improve performance

**Pre-normalization [GPT3].** To improve the training stability, we normalize the input of each transformer sub-layer, instead of normalizing the output. We use the RMSNorm normalizing function, introduced by Zhang and Sennrich (2019).

**SwiGLU activation function [PaLM].** We replace the ReLU non-linearity by the SwiGLU activation function, introduced by Shazeer (2020) to improve the performance. We use a dimension of $\frac{2}{3}4d$ instead of $4d$ as in PaLM.

**Rotary Embeddings [GPTNeo].** We remove the absolute positional embeddings, and instead, add rotary positional embeddings (RoPE), introduced by Su et al. (2021), at each layer of the network.

The details of the hyper-parameters for our different models are given in Table 2.

Source: LLama paper; Sebastian Raschka Twitter Thread.

# Some benchmarks: natural questions & code generation

| | | 0-shot | 1-shot | 5-shot | 64-shot |
|---|---|---|---|---|---|
| GPT-3 | 175B | 14.6 | 23.0 | - | 29.9 |
| Gopher | 280B | 10.1 | - | 24.5 | 28.2 |
| Chinchilla | 70B | 16.6 | - | 31.5 | 35.5 |
| PaLM | 8B | 8.4 | 10.6 | - | 14.6 |
| | 62B | 18.1 | 26.5 | - | 27.6 |
| | 540B | 21.2 | 29.3 | - | 39.6 |
| LLaMA | 7B | 16.8 | 18.7 | 22.0 | 26.1 |
| | 13B | 20.1 | 23.4 | 28.1 | 31.9 |
| | 33B | **24.9** | 28.3 | 32.9 | 36.0 |
| | 65B | 23.8 | **31.0** | **35.0** | **39.9** |

Table 4: **NaturalQuestions.** Exact match performance.

| | Params | HumanEval | | MBPP | |
|---|---|---|---|---|---|
| pass@ | | @1 | @100 | @1 | @80 |
| LaMDA | 137B | 14.0 | 47.3 | 14.8 | 62.4 |
| PaLM | 8B | 3.6* | 18.7* | 5.0* | 35.7* |
| PaLM | 62B | 15.9 | 46.3* | 21.4 | 63.2* |
| PaLM-cont | 62B | 23.7 | - | 31.2 | - |
| PaLM | 540B | **26.2** | 76.2 | 36.8 | 75.0 |
| LLaMA | 7B | 10.5 | 36.5 | 17.7 | 56.2 |
| | 13B | 15.8 | 52.5 | 22.0 | 64.0 |
| | 33B | 21.7 | 70.7 | 30.2 | 73.4 |
| | 65B | 23.7 | **79.3** | **37.7** | **76.8** |

Table 8: **Model performance for code generation.** We

- models out-perform GPT-3 with $< 10\%$ the parameters.

Source: LLama paper; Sebastian Raschka Twitter Thread.

# LLaMa 1, 2, 3

- LLaMa 1 released Feb 2023.
- newer variants:
  - LLaMa 2 (7B, 13B, 70B) released July 2023.
  - LLaMa 3 (8B, 70B) released April 2024.
  - LlaMa 3.1 (8B, 70B, 405B), released July 2024.
- LLaMa 2+ also have aligned versions (next lecture).

# “Open” LLMs

https://cameronrwolfe.substack.com/p/dolma-olmo-and-the-future-of-open

- OLMo and Apertus models are truly open, as they release their data, training/evaluation code, model weights, inference code, and more. Also, release on Apache 2.0 License.

# “Open” LLMs

https://cameronrwolfe.substack.com/p/dolma-olmo-and-the-future-of-open

- OLMo and Apertus models are truly open, as they release their data, training/evaluation code, model weights, inference code, and more. Also, release on Apache 2.0 License.
- LLaMa/DeepSeek: detailed documentation and open weights, but minimal information about the pretraining dataset.
- Mistral: model weights and a brief report.
- Falcon: partial subset of the model’s pretraining data, along with a report on the model and data.
- BLOOM: training code, model checkpoints, and training data, but license is restrictive.

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# Flash Attention

- used in the most recent models.
- hardware-aware (using super-fast SRAM rather than HBM).
- uses a trick to compute the attention operator's softmax denominator in chunks, without accessing the whole sequence.

# Rotary Positional Embeddings

- Standard positional embeddings:
  - learn vectors for each token position in the document, similar to learning vectors for each word in the vocabulary.
  - in the first tranformer block, add position embeddings to the inputted word embeddings.

# Rotary Positional Embeddings

- Standard positional embeddings:
  - learn vectors for each token position in the document, similar to learning vectors for each word in the vocabulary.
  - in the first tranformer block, add position embeddings to the inputted word embeddings.
  - for long context windows, doesn't work well because most training docs are too short to learn good embeddings. Also, lots of parameters to learn.

# Rotary Positional Embeddings

- Standard positional embeddings:
  - learn vectors for each token position in the document, similar to learning vectors for each word in the vocabulary.
  - in the first tranformer block, add position embeddings to the inputted word embeddings.
  - for long context windows, doesn't work well because most training docs are too short to learn good embeddings. Also, lots of parameters to learn.
- Rotary Positional Embeddings (RoPE, Su et al 2023):
  - rather than learn embeddings for token positions, rotate the query-token vector and key-token vector depending on the respective positions of the query token and the key token.

# Rotary Positional Embeddings

- Recall that for a two-dimensional vector $(x, y)$, the 2D rotation by angle $\theta$ is

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$

# Rotary Positional Embeddings

- Recall that for a two-dimensional vector $(x, y)$, the 2D rotation by angle $\theta$ is

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$

- Define query vector$q_i = W_Q x_i$ and key vector $k_j = W_K x_j$, each with dimensions $1, ..., n_E$
- For each odd-number dimension in the vectors: $l \in \{1, 3, 5, ..., n_E - 1\}$:
  - rotate the pair $(x, y) = (q_{i,l}, q_{i,l+1})$ and $(x, y) = (k_{j,l}, k_{j,l+1})$ by the angles

$$\theta_{i,l} = \frac{i}{\alpha^{2l/n_E}}, \theta_{j,l} = \frac{j}{\alpha^{2l/n_E}}$$

- Notes:
  - $\alpha = 10,000$ based on empirical tests (Vaswani et al 2017).
  - relative token position information encoded by differences in rotations between $\theta_i$ and $\theta_j$
  - for higher embedding dimensions (as $l$ increases), denominator shrinks and the embeddings rotate more slowly.

# GeGLU Activation Functions

Modern LLMs (starting with LLaMA) use **GeGLU as the activation function**, rather than ReLU, in the MLP segment of the transformer block.

- GeGLU = Gated Linear Unit (GLU) with a Gaussian Error Linear Unit (GELU) activation. SwiGLU (GLU with "Swish" activation) is similar.

# GeGLU Activation Functions

Modern LLMs (starting with LLaMA) use **GeGLU as the activation function**, rather than ReLU, in the MLP segment of the transformer block.

- GeGLU = Gated Linear Unit (GLU) with a Gaussian Error Linear Unit (GELU) activation. SwiGLU (GLU with "Swish" activation) is similar.
- GELU = Gaussian Error Linear Unit (GELU), a variant of ReLU that has the same advantages but is smooth/differentiable:

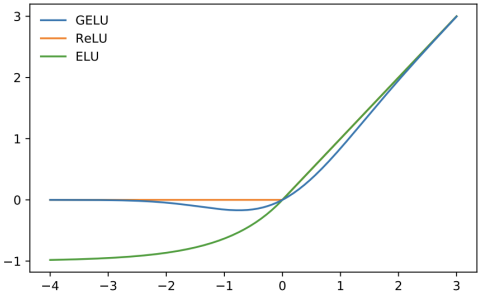



Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).

$\text{GELU}(x) = x\Phi(x)$, where $\Phi(\cdot)$ is the normal/Gaussian cdf.

# GeGLU Activation Functions

Modern LLMs (starting with LLaMA) use **GeGLU as the activation function**, rather than ReLU, in the MLP segment of the transformer block.

- GeGLU = Gated Linear Unit (GLU) with a Gaussian Error Linear Unit (GELU) activation.
  $\mathrm{GELU}(x) = x\Phi(x)$, where $\Phi(\cdot)$ is the normal/Gaussian cdf.
- GeGLU uses GELU as a gating function for each embedding dimension:

$$\mathrm{GeGLU}(x) = \mathrm{GELU}(x\mathbf{\Omega}_1) \odot (x\mathbf{\Omega}_2)$$

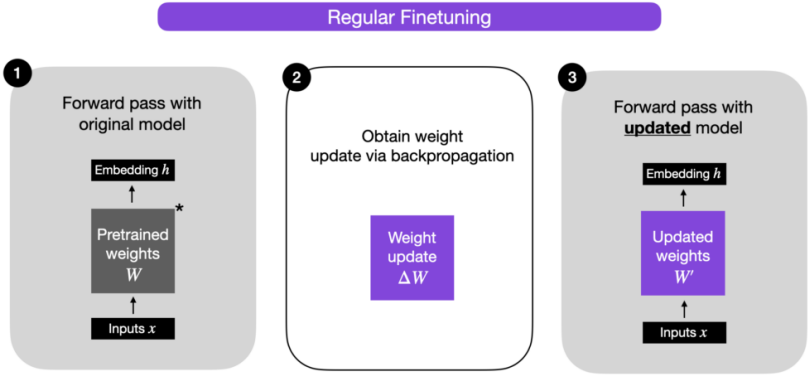
  - $\mathbf{\Omega}_1, \mathbf{\Omega}_2$ are learnable parameter matrices.
  - $\odot$ is element-wise multiplication across all dimensions of $x$.
- A more expressive "volume knob" compared to ReLU's "on/off switch":
  $\mathrm{ReLU}(x) = \max(0, x)$.

# LoRA: Low Rank Adaptation of Large Language Models

https://arxiv.org/abs/2106.09685 | https://github.com/microsoft/LoRA

https://lightning.ai/pages/community/tutorial/lora-llm/

- Fine-tuning large models with billions of parameters is extremely compute-intensive:

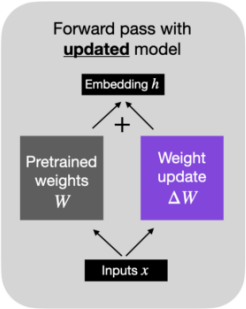



* The pretrained model could be any LLM, e.g., an encoder-style LLM (like BERT) or a generative decoder-style LLM (like GPT)

- yet fine-tuned models are using a tiny subset of an LM's generic capacity
  - →LoRA leverages this for much more efficient fine-tuning.

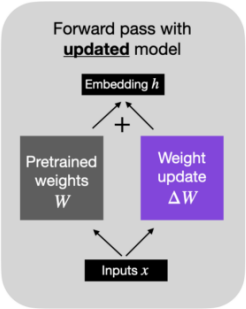
# Fine-tuning with LoRA

Alternative formulation (regular finetuning)

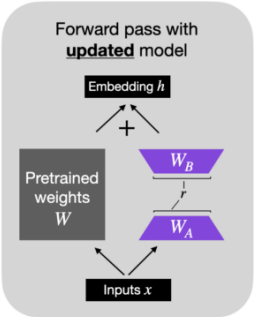


# Fine-tuning with LoRA

Alternative formulation (regular finetuning)




LoRA weights, $W_A$ and $W_B$, represent $\Delta W$




- For $n_A \times n_B$ layer weights $W$, define $\Delta W = W_A W_B$, where $W_A$ and $W_B$ are $n_A \times r$ and $r \times n_B$ lower-rank factor matrices.
  - $W$ is frozen and LoRA learns $W_A$ and $W_B$
  - the rank $r$ calibrates the level of compression; lower $r$ means more efficiency and more information loss.

# LoRA works well for specific tasks, but models learn general purpose abilities

| Model&Method | # Trainable Parameters | WikiSQL Acc. (%) | MNLI-m Acc. (%) |
|---|---|---|---|
| GPT-3 (FT) | 175,255.8M | **73.8** | 89.5 |
| GPT-3 (BitFit) | 14.2M | 71.3 | 91.0 |
| GPT-3 (PreEmbed) | 3.2M | 63.1 | 88.6 |
| GPT-3 (PreLayer) | 20.2M | 70.1 | 89.5 |
| GPT-3 ($\text{Adapter}^{\text{H}}$) | 7.1M | 71.9 | 89.8 |
| GPT-3 ($\text{Adapter}^{\text{H}}$) | 40.1M | 73.2 | **91.5** |
| GPT-3 (LoRA) | 4.7M | 73.4 | **91.7** |
| GPT-3 (LoRA) | 37.7M | **74.0** | **91.6** |

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# Seq2Seq Transformers

- The third category of transformer architectures, sequence-to-sequence (also called "text-to-text transformer"), includes both an encoder and a decoder.
  - encoder-only – vectorize a sequence to solve a single prediction task.
  - decoder-only – generate tokens to complete a sequence.
- encoder-decoder does both:
  - vectorize an input sequence into an encoding; then generate an output sequence based on the encoding and the newly generated tokens.

# Seq2Seq Transformers

- The third category of transformer architectures, sequence-to-sequence (also called "text-to-text transformer"), includes both an encoder and a decoder.
  - encoder-only – vectorize a sequence to solve a single prediction task.
  - decoder-only – generate tokens to complete a sequence.
- encoder-decoder does both:
  - vectorize an input sequence into an encoding; then generate an output sequence based on the encoding and the newly generated tokens.
- Useful, for example, for machine translation:

**EasyNMT**

- State-of-the-art machine translation with 3 lines of code
- Translation for 150+ languages
- Sentence & document translation
- Automatic language detection
- 4 pre-trained translation models:
  - opus-mt from Helsinki-NLP
  - mBART50 from Facebook AI Research
  - m2m_100 from Facebook AI Research (418M & 1.2B model)

```python
#Install via: pip install -U easynmt
from easynmt import EasyNMT
model = EasyNMT('opus-mt')

#Translate a single sentence to German
print(model.translate('This is a sentence we want to translate to German', target_lang='de'))

#Translate several sentences to German
sentences = ['You can define a list with sentences.',
             'All sentences are translated to your target language.',
             'Note, you could also mix the languages of the sentences.']
print(model.translate(sentences, target_lang='de'))
```

# Sequence-to-Sequence Models: BART, PEGASUS, T5, T5Gemma

These models use the machine-translation (encoder-decoder) transformer architecture, but both the input and output sequence are English.

# Sequence-to-Sequence Models: BART, PEGASUS, T5, T5Gemma

These models use the machine-translation (encoder-decoder) transformer architecture, but both the input and output sequence are English.

- BART = "Bidirectional Auto-Regressive Transformer"
  - works like a denoising autoencoder
  - adds noise to input sequence (e.g. BERT-masking, deleting, permuting), then tries to produce output sequence that is the original sequence.

# Sequence-to-Sequence Models: BART, PEGASUS, T5, T5Gemma

These models use the machine-translation (encoder-decoder) transformer architecture, but both the input and output sequence are English.

- BART = "Bidirectional Auto-Regressive Transformer"
  - works like a denoising autoencoder
  - adds noise to input sequence (e.g. BERT-masking, deleting, permuting), then tries to produce output sequence that is the original sequence.
- PEGASUS introduces Gap Sentence Generation:
  - input multiple sentences, mask out one sentence, and try to reconstruct the missing sentence.
  - works well as a summarization pre-training objective.

# Sequence-to-Sequence Models: BART, PEGASUS, T5, T5Gemma

These models use the machine-translation (encoder-decoder) transformer architecture, but both the input and output sequence are English.

- BART = "Bidirectional Auto-Regressive Transformer"
  - works like a denoising autoencoder
  - adds noise to input sequence (e.g. BERT-masking, deleting, permuting), then tries to produce output sequence that is the original sequence.
- PEGASUS introduces Gap Sentence Generation:
  - input multiple sentences, mask out one sentence, and try to reconstruct the missing sentence.
  - works well as a summarization pre-training objective.
- T5 is a large model (11B paramters) pre-trained to solve a set of language tasks (SuperGLUE), where the input includes instructions (e.g. "Summarize this sentence: ...").

# Sequence-to-Sequence Models: BART, PEGASUS, T5, T5Gemma

These models use the machine-translation (encoder-decoder) transformer architecture, but both the input and output sequence are English.

- BART = "Bidirectional Auto-Regressive Transformer"
  - works like a denoising autoencoder
  - adds noise to input sequence (e.g. BERT-masking, deleting, permuting), then tries to produce output sequence that is the original sequence.
- PEGASUS introduces Gap Sentence Generation:
  - input multiple sentences, mask out one sentence, and try to reconstruct the missing sentence.
  - works well as a summarization pre-training objective.
- T5 is a large model (11B paramters) pre-trained to solve a set of language tasks (SuperGLUE), where the input includes instructions (e.g. "Summarize this sentence: ...").
- T5Gemma is a sequence-to-sequence model distilled from the decoder-only Gemma.

# Outline

Language Modeling

Transformer-based Language Models

Autoregressive Transformers (e.g. GPT)
- GPT Architecture/Training
- Open-Source Autoregressive LLMs

Additional Nuts and Bolts

Encoder-Decoder (Sequence-to-Sequence) Transformers

Interviews

# Easy Interviews

Elliott Ash
ETH Zurich

Soumitra Shukla
Harvard Business School

Jason Sockin
Cornell University

AEAs 2026

# Report on a recent job interview

**Please adhere to the Code of Conduct**
We review and approve each post based on our Code of Conduct before it is published on the website.

Employer *

IZA

Rate the experience overall*

*Positive*

Job title *

Postdoctoral Scholar

Describe the interview process*

test test test

# LLM Prompts for Colleague Ability and Accepting Anyone

I will provide you with a text snippet that describes a job interview.

---

**Colleague ability.** Does the description say that prospective colleagues are highly able, intelligent, or competent? Determine whether the interview description explicitly mentions or strongly implies that colleagues at the employer are of high ability, intelligence, or competence.

**"They'd accept anybody".** Does the description suggest, through explicit mention or clear implication, that the employer would hire anyone? That is, implying that the employer is desperate for employees and would accept any applicant regardless of qualifications or fit.

---

Read the description and provide a label "Definitely Yes", "Probably Yes", "Probably Not", "Definitely Not", or "N/A", followed by a semicolon (;) and then a short one-sentence explanation.

# Applying a Large Language Model (LLM) to Text

Ask Gpt-4o-mini to annotate a sample of interview processes

- 500 interviews for each industry x difficulty level
- Using these as a training set, scale up to all interviews using RoBERTa model

Then do human inspection and qualitative evaluation to validate output from LLM.

- LLM annotations seem to compare well to human judgment
  - Ask 2 humans to annotate a sample of prompts
  - The two agree with GPT $\sim$ 75-85% of the time
    - Same level of agreement as between humans

# Easy Interviews → Perceive Lower-Ability Colleagues

Accurate beliefs? Covariates Engaging? RJP CGQ HC

50% ↓ mention high-ability colleagues, *2.5x-to-6x* ↑ belief firm hires anyone

| | Cross-section sample | | Within-worker sample | |
|---|---|---|---|---|
| **Panel A: DV = 1(High-ability colleagues)** | | | | |
| 1(Easy interview) | -0.020*** | | -0.035*** | -0.025*** |
| | (0.001) | | (0.003) | (0.006) |
| Fraction others' interviews easy | | -0.022*** | | |
| | | (0.002) | | |
| Mean DV for not easy | 0.037 | 0.037 | 0.054 | 0.054 |
| **Panel B: DV = 1(Would accept anybody)** | | | | |
| 1(Easy interview) | 0.061*** | | 0.043*** | 0.028*** |
| | (0.001) | | (0.003) | (0.005) |
| Fraction others' interviews easy | | 0.074*** | | |
| | | (0.003) | | |
| Mean DV for not easy | 0.012 | 0.012 | 0.011 | 0.011 |
| N | 421,112 | 421,112 | 20,627 | 20,627 |
| Worker FE | | | | ✓ |

# Labeling Documents with LLMs

# Labeling Documents with LLMs

- For a labeling task where humans get high agreement, LLMs provide very good performance.
  - my sense: performance is comparable to attentive humans since GPT-4o (May 2024).
  - use the API (Application Programming Interface) and annotate documents programmatically from Python or R (highly parallelizable).
  - set temperature = zero so that outputs are (almost) reproducible.

# Labeling Documents with LLMs

- For a labeling task where humans get high agreement, LLMs provide very good performance.
  - my sense: performance is comparable to attentive humans since GPT-4o (May 2024).
  - use the API (Application Programming Interface) and annotate documents programmatically from Python or R (highly parallelizable).
  - set temperature = zero so that outputs are (almost) reproducible.
- For a labeling task where humans do not agree, LLMs are not a magical solution to that.

# Labeling Documents with LLMs

- For a labeling task where humans get high agreement, LLMs provide very good performance.
  - my sense: performance is comparable to attentive humans since GPT-4o (May 2024).
  - use the API (Application Programming Interface) and annotate documents programmatically from Python or R (highly parallelizable).
  - set temperature = zero so that outputs are (almost) reproducible.

- For a labeling task where humans do not agree, LLMs are not a magical solution to that.
  - if the task is too difficult for your labelers, try to get better labelers. If LLM outputs cannot be externally validated, they can't be relied on.

# Labeling Documents with LLMs

- For a labeling task where humans get high agreement, LLMs provide very good performance.
  - my sense: performance is comparable to attentive humans since GPT-4o (May 2024).
  - use the API (Application Programming Interface) and annotate documents programmatically from Python or R (highly parallelizable).
  - set temperature = zero so that outputs are (almost) reproducible.

- For a labeling task where humans do not agree, LLMs are not a magical solution to that.
  - if the task is too difficult for your labelers, try to get better labelers. If LLM outputs cannot be externally validated, they can't be relied on.

  - if the task is not well-defined, or the information is not contained in the document, LLMs will not provide a useful answer.